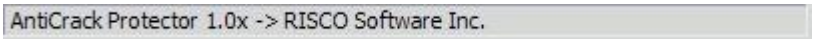
Programa:	Cool Color Picker 1.1	
Protección:	ACProtect 1.x+ Trial 10 días	
Objetivos:	Desempakar y evitar la limitación temporal	
URL descarga:	www.color-picker.net	
Dificultad:	Para nada difícil, si lo hice yo…	
Herramientas:	OllyDbg + PEiD + Import Reconstructor.	
Cracker:	SyXe'05 [hAcK-c0d3d] [CracksLatinos]	Fecha: 25-08-2005

Hola a todo el mundo antes de comenzar..!!

Nuestra presa de hoy será un programa empaquetado con ***ACProtect 1.0x*** según ***PEiD*** o ***1.32*** según el ***RDG***, igual nos da esto pues si nos ponemos nos ponemos y la versión no debe importar mucho pienso yo, antes de nada decir que una única vez me he enfrentado a este packer abordándolo desde nuestro querido ***SICE*** me fue imposible desempakarlo, jeje.. gajes del oficio, espero poder terminar el tuto de hoy, hace 3 días que no escribo nada he estado muy liado con un ***SafeDisc*** que no veas como se las gasta si alguien me echa una mano quiza podamos, jeje.. lo digo en serio se trata de las nuevas versiones de ***Adobe*** que están en su web, la mitad vienen con el inoperante ***Vbox*** y otras cuantas con este ***SafeDisc*** que lo que me despista es como maneja algunas ***dlls*** en la ***IAT***, bueno si alguien se anima aquí estaré que me escriba a syxe00@yahoo.es como siempre.

Y dicho esto pasemos al cracking que es a lo que hemos venido, la verdad es que son las 23:07 de la noche y hoy llevo un día muy duro, espero terminar esto, a ver veamos con el ***PEiD*** como siempre a ver que nos dice:



decir que insertaré el mínimo número de imágenes ya que sinó ***+NCR*** se me queja del peso del arhivo, jeje.. saludos compañero insertaré pues las esenciales con el tamaño justo, venga pues ***AntiCrack Protector 1.0x*** es nuestro próximo objetivo, en concreto desempakar el archivo pero no nos detendremos ahí sinó que lo vamos a crackear ya de paso, el que me vaya conociendo sabe que si empiezo algo lo termino, algún día publicaré trabajos de desempakados que no ven la luz porque no he resuelto el ***Trial*** o limitaciones, es una lástima pero lo veo así no me gusta andar a medias en las cosas.

Comienza el desempaquetado de ACProtect 1.0x-1.32…

Debo de reconocer que cuando vi el nombre me asusté un poco, jeje.. y quien no verdad? incluso pregunté a ***Ricardo*** si conocía el packer ya que ni si quiera lo relacionaba con ***ACProtect***, claro ***AntiCrack Protector***, ahora visto así parece tontería pero realmente me desconcertó, tras conocer de quien se trataba investigué un poco, leí algunos tuts de cómo otros crackers de la lista lo abordaron, siempre como recalco una y mil veces es ***IMPORTANTE RECABAR INFORMACIÓN***!! No podemos ir por la vida como unos indocumentados que nos llevamos las collejas, sino todo lo contrario, como se suele decir hay que conocer al "enemigo" al máximo.

Pues en marcha, si cargamos el programa en ***Olly*** después de parar en una ***INT 1*** se va todo al carajo, es realmente espantosa la manera en que nos cierra el ***Olly***, aquí comienzan los problemas, no es el típico packer que se deja tracear y tracear hasta la saciedad que incluso llegas al salto al ***OEP*** con ligera comodidas, de eso nada todo lo contrario, por lo visto nos detecta y ***PLUM***¡! Se cierra, jeje.. genial pues vale, es muy similar a lo que ocurre con algunas versiones más nuevas de Armadillo con las llamadas a ***OutputDebug***.. pero sin realizar la llamada ¿cómo lo hará? Pensé yo en un primer momento, ***GetTickCount***? Lo probé y nada, el ***GlobalFlag***? Parado en el ***EP*** del ***packer*** me fui a la ventana de dump con ***Ctrl+G*** puse ***FS:[30]+68*** + ***Enter*** y cambié el ***0x70*** a ***0x00*** a ver si era eso pero tampoco, tengo ***Olly*** parcheado con ***IsDebug*** y sin nombre en la ***TitleBar*** así pues quedan descartadas estas dos protecciones también, aquí es cuando recordé haber leído que miraba el ***handle*** de la aplicación que lo había lanzado y de no ser el ***Explorer*** que es lo normal se cierra ya que evidentemente se percata de que está siendo depurado, antes de continuar os quiero mostrar lo que encontré en la Ayuda del compresor ***ACProtect 1.0***, jejeje.. tandrá todo esto que dicen? Veamos:

Embedded Protector - You can specify the place in your application to embed the inner cryptor. With the embedded cryptor, your application can not be cracked even though the cracker know the original entry point (OEP) when rebuilding your Import table.

Code Replace - Acprotect will replace some of your program 's code with the same function code in the ACProtect codes,If the ACPrtect code are stripped ,then the codes replaced will not function normally.

Metamorph - Metamorph decryption routines and several encrypted layers. The code being protected varies every time you protect your application.

RSA 1024 - With RSA keys system a cracker can not write a KeyGen (or KeyMaker) for your product without the RSA private key (the RSA private key is only in the author's hand). You may also specify some code to be locked by the RSA keys, the cracker won't have the proper "key.dat" on the proper machine, without it the protected code can not decrypt.

Mutual Communication - API system which allows communication between application and loader, loader can call some of the application's export function, and the application can call the function the protected system provided.

ShareIt Keygen Program Auto Made- ACProtect can generate ShareIt.com compatible keygen program to fill shareware author's need. The Keygen program can generate registration Codes,and ShareIt.com will send it to registered customer.

nti Patch/Loader - Protects your files against patching/modifications and disassembling.

Anti-SoftIce - Numega SoftIce (tm) detection and Frogsice detection

Anti Debug/Dump/Trace - Anti-debugger & Monitor like W32Dasm, SoftIce, TRW 2000, Turbo Debugger, Sourcer, Filemon, ExeSpy, ResSpy, RegMon, Memory Monitor etc.

Runtime Code Encryption - Important parts of your code can be protected individually, providing extra protection against memory patches.

System-Specific Licenses - License keys may be locked to a specific system by checking the computer's unique fingerprint (such as Harddisk ID, CPU ID etc.)

Stolen keys - Keeping of the database and checkup of "stolen" (illegal) registration keys.

Trial setting - Possibility to create evaluation (trial) versions, that limit application functions based on the number of runs left . number of days left or deadline limit.

No veas no? Según dicen tiene de todo, quizá lo hacen para asustar ¿? Bueno pues nada sigamos nuestro caminito que andando se llega a Roma chavales, el que quiera más información que lea las ***teorías 160***, ***161***, ***162***, ***165***, ***170***, ***171*** de nuestro pedazo de grupo que no dejan lugar a dudas de cómo derrotar a esta pieza, luego hay algún texto más de ***+NCR*** que yo recuerde, algunos de ***Shoulck*** y supongo que de otros miembros también pero no los he visto o no me acuerdo, las teorías citadas creo recordar que son de ***Ricardo***. Y ahora sí vamos a resolver esto, primero deberemos conseguir que corra en ***Olly***, utilizaremos un método que cuando lo vi me llamó mucho la atención y es de ***Shoulck*** o almenos creo habérselo visto hacer a él, también a ***Ricardo*** por eso y es el siguiente: Cargamos el programa en ***Olly*** y le hacemos un loop infinito en el ***EP*** así:

```
Address   Hex dump   Disassembly                          Comment
0050F000  - EB FE    JMP SHORT ColorPic.<ModuleEntryPoint>
0050F002    F8       CLC
0050F003    4F       DEC EDI
0050F004    87CF     XCHG EDI,ECX
```

Y guardamos el archivo como ***ColorPicker_loop.exe*** o como nos de la gana, yo lo pondré así para tenerlo a la vista rápido, salvamos el archivo, supongo que todos sabemos hacer eso, y lo ejecutamos desde el ***Explorer*** que es lo que el quería no? Ahora lo atacheamos desde otro ***Olly*** le deshacemos el bucle y a correr, si alguien quiere investigar el motivo pues ya sabe que ponga un ***HE*** en ***Process32Next*** e irá viendo como va mirando los procesos cargados y lo interesante es que compara su handle con el del programa que lanzó al propio ***ACProtect***, para no tener problemas se puede nopear este salto:

```
Address   Hex dump      Disassembly                          Comment
00513A3B  v 0F84 EF010000  JE ColorPic.00513C30
00513A41    8B95 55414000  MOV EDX,DWORD PTR SS:[EBP+404155]
00513A47    3B95 45414000  CMP EDX,DWORD PTR SS:[EBP+404145]
00513A4D  v 0F84 DC000000  JE ColorPic.00513B2F
00513A53    B8 4D414000    MOV EAX,ColorPic.0040414D
00513A58    03C5           ADD EAX,EBP
```

Si lo dejamos bien nopeadito no dará más problemas, ah! Otra cosa como tengáis algún ***driver*** pululando por la memoria de algún programa de cracking también se cerrará, ejemplo:

```
0012FF40  00514296  CALL to CreateFileA from ColorPic.00514290
0012FF44  005144D5  FileName = "\\.\SICE"
0012FF48  C0000000  Access = GENERIC_READ|GENERIC_WRITE
0012FF4C  00000003  ShareMode = FILE_SHARE_READ|FILE_SHARE_WRITE
0012FF50  00000000  pSecurity = NULL
0012FF54  00000003  Mode = OPEN_EXISTING
0012FF58  00000080  Attributes = NORMAL
0012FF5C  00000000  hTemplateFile = NULL
0012FF60  0051B11F  ColorPic.0051B11F
```

Se ve no? Está buscando si está cargado el ***SoftIce***, hace lo mismo con mogollón de ellos, poner un ***HE*** en ***CreateFileA*** y los iréis viendo todos, si por casualidad estáis trabajando con alguno de estos programas jugar con los saltos posteriores a la llamada, sino no os preocupéis no hay problema de que preocuparse. Un ejemplo si tenéis el ***Regmonitor*** cargdo el retorno será ***EAX==4*** que deberéis cambiar por ***EAX==FFFFFFFF*** (-1) y listos!! Mira también además que no tengamos ***BP's*** tipo ***INT 3*** en algunas ***APIs*** comunes, ejemplo:

```
:00514352   8B85 19444000   MOV EAX,DWORD PTR SS:[EBP+404419]      ; kernel32.CloseHandle
:00514358   8038 CC         CMP BYTE PTR DS:[EAX],0CC  <=Aquí!!
:0051435B   74 10           JE SHORT ColorPic.0051436D
```

Busca un byte '***CC***' que es el ***INT3*** precisamente, si son iguales es que hay un ***BP*** puesto y salta a eror

Que no acabamos nunca va, ejecutamos el exe que tiene el loop en el ***EP***, quedará suspendido, lo atacheamos con un ***Olly***, le deshacemos el loop deberéis haber anotado los bytes que reemplazamos que eran ***60F8*** sinó no pasa nada lo miráis del proceso original, damos ***RUN*** y para en una ***INT 1***, nada ***Shift+F9*** y sale el programa, vaya repetimos la jugada pero tras la excepción ponemos un ***BPM*** en la primera sección, caremos aquí:

```
Address   Hex dump        Disassembly                          Comment
0041A4C9  68 28F34300     PUSH ColorPic.0043F328
0041A4CE  68 58DC4100     PUSH ColorPic.0041DC58
0041A4D3  64:A1 00000000  MOV EAX,DWORD PTR FS:[0]
0041A4D9  50              PUSH EAX
0041A4DA  64:8925 000000  MOV DWORD PTR FS:[0],ESP
0041A4E1  83EC 58         SUB ESP,58
0041A4E4  53              PUSH EBX
0041A4E5  56              PUSH ESI
0041A4E6  57              PUSH EDI
0041A4E7  8965 E8         MOV DWORD PTR SS:[EBP-18],ESP
0041A4EA  FF15 50B24300   CALL DWORD PTR DS:[43B250]           kernel32.GetVersion
0041A4F0  33D2            XOR EDX,EDX
```

Está claro que este es el ***OEP*** y que es un ejecutable compilado en ***C++*** ya que el código nos habla, sabemos además que la estructura inicial de este compilador es siempre la misma y como dice ***Akira*** nos da gran información que debemos interpretar, en este caso los ***Stolen Bytes*** son facilísimos, vamos que para hacer esto mejor no hacer nada lo digo en serio, veamos el inicio típico en ***C++***

```
PUSH EBP
MOV EBP, ESP
PUSH -1
PUSH xxxxxxxx
PUSH xxxxxxxx
. . .
```

Contamos que nos faltan exactamente ***5 bytes***, luego los stolen serán:

```
Address   Hex dump   Disassembly   Comment
0041A4C4  55         PUSH EBP
0041A4C5  8BEC       MOV EBP,ESP
0041A4C7  6A FF      PUSH -1
```

Resuelto este tema pasemos a otro, si hubieran sido más bytes o incluso si hubiera sido más complicado obtenerlos siempre tenemos el método que utilizamos en ***ASProtect*** para obtenerlos pero en este caso es una pérdida absoluta de tiempo, sigamos.. tenemos el ***OEP*** y los Stolen, vamos a por la ***IAT***, vemos bajo el ***OEP*** un salto a ***GetVersion*** hacemos ***Follow in dump->Memory Address*** y estamos en la ***IAT***, buscamos su inicio que encontramos fácilmente ya que es el inicio de la sección ***.rdata***, o sea:

Inicio IAT: 43B000

Hacemos lo mismo para el final de la misma que tampoco tiene dificultad ninguna:

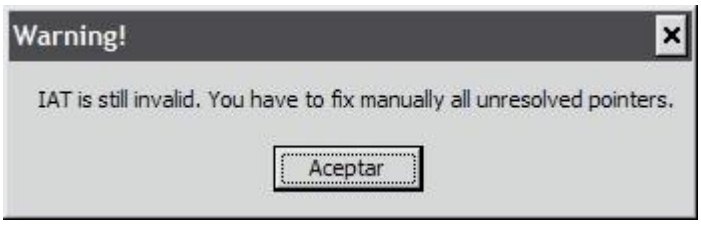
Final IAT: 43B634

Por seguridad siempre le doy un pequeño margen al final, vamos con estos datos al Import Reconstructor:

Picamos en el botón ***Get Imports*** y nos dice que quedó un puntero por resover, bueno sólo uno:

IATrva: 3B4D0
Apunta a: 5124DB
Api correcta: ¿?

La entrada queda dentro del rango de punteros del ***User32.dll*** lo que induce a pensar que será una api de esta librería pero si miramos detenidamente la entrada veremos que no apunta a ninguna api ni emulada ni nada es como una especie de rutina extraña ahí puesta para molestar quizá o para despistar ¿? Sea como sea ignoramos esto de momento, vamos a dumpear, nos situamos en el ***PUSH EBP*** de los stolen y le hacemos un ***New origin here*** para situar allí el ***EIP*** debe quedar en ***:41A4C4***, y dumpeamos con el propio ***Ollydump***, a mi personalmente me va muy bien mejor que ***LordPe*** diría y todo pero es una opinión perosonal, cuando tengamos el dumpeado le hacemos un ***Fix Dump*** desde el Import para dejarlo listo ignorando el siguiente mensaje que aparecerá:



Ni caso, aceptamos y quedará el archivo reparado, de hecho si lo ejecutáis funciona bien pos lo que hemos terminado el desempakado, la verdad que mucha publicidad de packer y luego no es gran cosa, pero bueno esto se dice a toro pasado, he de reconocer que mis problemas tuve en una intentona hace tiempo y salí bien escaldado, jejeje.. ahora a por las limitaciones!!

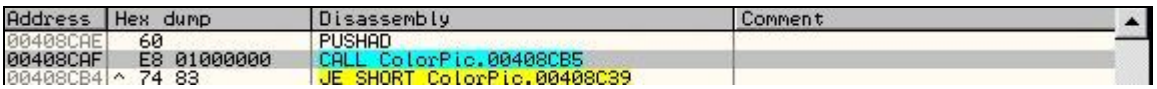
Registrando el producto…

Cuando ejecutamos el programa vemos esta nag que nos informa que es la versión ***Trial*** del producto y que además tenemos sólo ***10 días*** de prueba ¡un poco tacaños si son!

He de decir que es el primer ya día que lo acabo de instalar, así esos 10 días son los totales que da, no da más ***;-(*** pero bueno para nosotros será suficiente, jajaja… con ***1h*** nos conformamos verdad?

Cargamos el dumpeado ya reparado, con stolen y tabla de importaciones correcta en ***Olly***, ahora no debemos preocuparnos por el tema de que nos lo cierre ya que eso era una fechoría del compresor y no del programa en sí, ahora trabajaremos con el desempaquetado sin ***ACProtect*** ya, entonces si encontramos un lugar donde parchear podremos hacerlo, ver sus cadenas, desensamblarlo, etc., en fin todo lo que podemos hacer con cualquier programa que esté sin comprimir.

Comenzamos a tracear y ya vemos cosas raras, atención a esto que es muy grave, estamos aquí antes de pasar por el call que hay y que se puede ver en la imagen:



Aparentemente un ***CALL*** normal y de hecho lo es, lo extraño viene luego cuando lointentamos superar con ***F8***:

```
INT3 command at ColorPic.0040AED1
```

Nos dice que hay un ***INT3*** en ***:40AED1*** pero si nosotros no pusimos ninguno ¿? Seguramente será un error o algo pensé al principio pero si vamos a la direccion que marca el ***Olly*** vemos esto:

```
Address   Hex dump     Disassembly                  Comment
0040AED1   CC          INT3
0040AED2   90          NOP
0040AED3   64:67:8F06 000( POP DWORD PTR FS:[0]
0040AED9   83C4 04     ADD ESP,4
0040AEDC   60          PUSHAD
0040AEDD   E8 00000000 CALL ColorPic.0040AEE2
```

Pues si es verdad y todo ya empezaba hasta a dudarlo, y el color rojo de las instrucciones!! Es como si se hubieran modificado que cambian a ese color, pero miremos mejor la zona:

```
Address   Hex dump     Disassembly                  Comment
0040AE9A   E8 2A000000 CALL ColorPic.0040AEC9
0040AE9F   8B4424 04   MOV EAX,DWORD PTR SS:[ESP+4]
0040AEA3   8B4C24 0C   MOV ECX,DWORD PTR SS:[ESP+C]
0040AEA7   FF81 B8000000 INC DWORD PTR DS:[ECX+B8]
0040AEAD   8B00        MOV EAX,DWORD PTR DS:[EAX]
0040AEAF   2D 03000080 SUB EAX,80000003
0040AEB4   75 12       JNZ SHORT ColorPic.0040AEC8
0040AEB6   90          NOP
0040AEB7   90          NOP
0040AEB8   90          NOP
0040AEB9   90          NOP
0040AEBA   33C0        XOR EAX,EAX
0040AEBC   8941 04     MOV DWORD PTR DS:[ECX+4],EAX
0040AEBF   8941 08     MOV DWORD PTR DS:[ECX+8],EAX
0040AEC2   8941 0C     MOV DWORD PTR DS:[ECX+C],EAX
0040AEC5   8941 10     MOV DWORD PTR DS:[ECX+10],EAX
0040AEC8   C3          RETN
0040AEC9   33C0        XOR EAX,EAX
0040AECB   64:FF30     PUSH DWORD PTR FS:[EAX]
0040AECE   64:8920     MOV DWORD PTR FS:[EAX],ESP
0040AED1   CC          INT3
0040AED2   90          NOP
```

No lo he comprobado pero parece la estructura típica del manejador de escepciones (***SE handler***) mirad como hace un ***SUB EAX, 80000003*** no os suena este valor de Armadillo? Si claro de los Nanomites, está mirando si se produjo una ***INT3*** cuyo código asociado es precisamente este ***80000003*** que es el cartel que muestra un ***Armadillo*** que encuentra un ***Nanomite*** y no tiene padre, lo que es más delator es que después del salto en caso de no saltar se hace un ***XOR EAX, EAX*** poniendo el registro a ***0*** y luego se pasa a cuatro variables, variables?? Pero sin son ***DRO***, ***DR1***, ***DR2*** y ***DR3***..!! aqui nos borra los ***HE*** el condenao, está claro que es el ***SE handler***, luego lo que está pasando es que o bien que nos detecta y nos tira fuera deliberadamente o que se produce una excepción que no puede manejar por algún motivo, no hace falta que os diga que llegados a este punto las posibilidades de salir son bajo mínimas sino intentarlo y lo veréis, tocar ***F7*** o ***F8*** y caeréis en ***RaiseException*** al momento, entonces? Que hacer, como lo solventamos?

Debida a la poca experiencia que uno tiene todavía y visto lo visto se meocurrió no superar ningún ***CALL*** con ***F8*** y creedme que funciona, no sé el porqué con ***F8*** no y con ***F7*** si, alguna explicación? Yo la desconozco si alguien la sabe pues nos podría orientar, la cuestión es que del ***F8*** nos olvidamos, seguiremos desde aquí con ***F7*** ¿ok?

Siguiendo este método no es difícil darse cuenta que para entrar sin la ***nag*** debería saltar aquí:

```
Address   Hex dump     Disassembly                  Comment
0040BF0A   FFD6        CALL ESI                     Apunta a :5124DB
0040BF0C   8A85 E8FEFFFF MOV AL,BYTE PTR SS:[EBP-118]
0040BF12   6A 00       PUSH 0
0040BF14   84C0        TEST AL,AL
0040BF16   0F85 D0000000 JNZ DUMP_inj.0040BFEC
0040BF1C   8D8D 80EAFFFF LEA ECX,DWORD PTR SS:[EBP-1580]
0040BF22   E8 894F0000 CALL DUMP_inj.00410EB0
```

Vale de momento reiniciamos con ***Ctrl+F2*** y desde el ***OEP*** pondremos un ***BP*** en ***:40BF0A***, vamos para allí:

```
Address   Hex dump     Disassembly                  Comment
0040BF0A   25 504AEF73 AND EAX,73EF4A50             Apunta a :5124DB
0040BF0F   27          DAA
0040BF10   24 1B       AND AL,1B
0040BF12   C9          LEAVE
```

Queeee? Donde está el ***CALL*** que allí había, esto es alucinante si allí mismo había un ***CALL ESI*** hace un momento justo antes de reiciciar, incluso vemos el comentario que pusimos al lado lo que denota claramente que no nos estamos equivocando de dirección, a estas alturas se me ocurrió pensar que estaba encriptado el código!! Entonces deberíamos esperar a que se desencipte para meterle mano, ahí está el tema, de momento nos situamos

sobre la instrucción ***:40BF0A***, hacemos un ***Follow in Dump->Selection*** y le ponemos un ***BMP On Write*** para ver cuando y donde desencripta:

```
Address   Hex dump     Disassembly                          Comment
0040BDB8    8928       MOV DWORD PTR DS:[EAX],EBP
0040BDBA    50         PUSH EAX
0040BDBB    E8 01000000 CALL DUMP_inj.0040BDC1
0040BDC0  v 77 58      JA SHORT DUMP_inj.0040BE1A
```

Aquí el amigo está desencriptando código nada de ***WriteProcessMemory*** sino dword a dword, bouuuuuff, está en un loop donde va desencriptando el bloque, parece ser que ***EBP*** va tomando cuerpo con los bytes desencriptados, se puede apreciar que la zona desencriptada es bastante grande, poner un ***BPM On Write*** más abajo y veréis lo que os digo, en concreto son dos zonas en estos intervalos:

de :408E0E hasta :40B80C /* primero en desencriptarse desde :408DA6 y 408EF1 y :40904A finalmente*/
de :40B9F3 hasta :40C020 /* desencripatado después desde :40904A */

Estos son los dos que hay o que yo vi pero luego le descubrí algunos lugares más desde donde desencripta como:

```
Address   Hex dump     Disassembly                          Comment
00408EF1    8937       MOV DWORD PTR DS:[EDI],ESI
00408EF3  v EB 01      JMP SHORT DUMP_inj.00408EF6
00408EF5  ^ 79 E9      JNS SHORT DUMP_inj.00408EE0
```

```
Address   Hex dump     Disassembly                          Comment
0040904A    8928       MOV DWORD PTR DS:[EAX],EBP
0040904C    50         PUSH EAX
0040904D    E8 01000000 CALL DUMP_inj.00409053
```

En total descubrí ters puntos donde va desencriptando, pero lo interesante es que estos dos caen dentro del primer bloque que se desencriptó así que al igual que el ***CALL ESI*** estará encriptado desde el inicio, que lo hace desde ***:408DA6*** que a su vez está también encriptado y se desencripta desde aquí:

```
Address   Hex dump     Disassembly                          Comment
0040BC51    8955 00    MOV DWORD PTR SS:[EBP],EDX
0040BC54  v EB 01      JMP SHORT DUMP_inj.0040BC57
0040BC56  v 74 0F      JE SHORT DUMP_inj.0040BC67
```

jeje y podríamos seguir así, pues esta zona se desencripta desde aquí:

```
Address   Hex dump     Disassembly                          Comment
00408DA6  > ↳890B      MOV DWORD PTR DS:[EBX],ECX
00408DA8  .v EB 01     JMP SHORT DUMP_inj.00408DAB
00408DAA    E8         DB E8
```

Y esta zona si que está accesible desde el ***OEP*** así que es el primer lugar que para a desencriptar, en total hemos visto ***5*** lugares donde va desencriptando código en tiempo de ejecución que luego ejecutará, recordad que sabemos que salto jumpear para entrar sin la nag pero está lógicamente encriptado de inicio, a su vez la rutina que desencripta está encriptada, jejeje.. y así en ***5*** niveles de ahí los ***5*** lugares que descubrimos, lo que sabemos seguro es donde empieza y es en ***:408DA6*** como se ve en la última imagen, si reiniciais ***Ctrl+F2*** y hacéis un ***Go*** con ***Ctrl+G*** a esta dirección veréis que el código está accesible ya lo que deja entrever que no se debe desencriptar, esto es lógico, por muchas rutinas que tenga almenos una debe estar desencriptada que es digamos la que inicia la operación que en nuestro caso está en ***:408DA6***

Tiene otras más en otros lugares que son estas::

```
Address   Hex dump     Disassembly                          Comment
0040B97C    8930       MOV DWORD PTR DS:[EAX],ESI
0040B97E    E8 01000000 CALL DUMP_pr.0040B984
0040B983  ^ EB 83      JMP SHORT DUMP_pr.0040B908
```

```
Address   Hex dump     Disassembly                          Comment
0040BAD2  .   8932     MOV DWORD PTR DS:[EDX],ESI
0040BAD4  .v EB 01     JMP SHORT DUMP_pr_.0040BAD7
0040BAD6    7A         DB 7A                                CHAR 'z'
```

Finalmente son ***7 rutinas desencriptadoras*** que encontré y creo que no hay más, esto lógicamente es un truquito para que no modifiquemos los bytes que queramos claro, además me llamó mucho la atención una de estas rutinas porque mirad que prepara en ***:40904A***, jejeje:

```
0040968E                    45 58 45 53 50 59 00        EXESPY.
0040969E  57 58 52 39 35 00 52 45 47 4D 4F 4E 00 46 49 4C  WXR95.REGMON.FIL
004096AE  45 20 4D 4F 4E 49 54 4F 52 00 52 45 47 4D 4F 4E  E MONITOR.REGMON
004096BE  45 58 00 57 49 4E 44 4F 57 20 44 45 54 45 43 54  EX.WINDOW DETECT
```

```
004096CE  49 56 45 00 44 45 42 55 47 56 49 45 57 00 52 45  IVE.DEBUGVIEW.RE
004096DE  53 53 50 59 00 41 44 56 41 4E 43 45 44 20 52 45  SSPY.ADVANCED RE
004096EE  47 49 53 54 52 59 20 54 52 41 43 45 52 00 52 45  GISTRY TRACER.RE
004096FE  47 53 4E 41 50 00 4D 45 4D 53 50 59 00 4D 45 4D  GSNAP.MEMSPY.MEM
0040970E  4F 52 59 20 44 4F 43 54 4F 52 00 50 52 4F 43 44  ORY DOCTOR.PROCD
0040971E  55 4D 50 33 32 00 4D 45 4D 4F 52 59 20 45 44 49  UMP32.MEMORY EDI
0040972E  54 4F 52 00 46 52 4F 47 53 49 43 45 00 53 4D 55  TOR.FROGSICE.SMU
0040973E  20 57 49 4E 53 50 45 43 54 4F 52 00 4D 45 4D 4F   WINSPECTOR.MEMO
0040974E  52 59 20 44 55 4D 50 45 52 00 4D 45 4D 4F 52 59  RY DUMPER.MEMORY
0040975E  4D 4F 4E 49 54 4F 52 00 4E 55 4D 45 47 41 20 53  MONITOR.NUMEGA S
0040976E  4F 46 54 49 43 45 20 4C 4F 41 44 45 52 00 55 52  OFTICE LOADER.UR
0040977E  53 4F 46 54 20 57 33 32 44 41 53 4D 00 47 45 4E  SOFT W32DASM.GEN
0040978E  45 52 49 43 5F 57 4B 54 54 45 4C 4F 43 4B 44 55  ERIC_WKTTELOCKDU
0040979E  4D 50 45 52 00 2D 3D 43 48 49 4E 41 20 43 52 41  MPER.-=CHINA CRA
004097AE  43 4B 49 4E 47 20 47 52 4F 55 50 3D 2D 00 4F 6C  CKING GROUP=-.Ol
004097BE  6C 79 44 62 67 00 54 52 57 32 30 30 30 00 00 00  lyDbg.TRW2000...
```

el nombre de aplicaciones de ***cracking***… la verdad que ya había oído hablar de este truco pero como nunca lo encontré me hizo gracia verlo, esto si os interesa lo escribe la rutina desencriptadora ***:40904A*** como dije antes, esto por si alguien está interesado en saber más, lo que hace pues no hace falta que diga lo que busca no? Aunque no sé pero me da que no chuta porque tengo abierto un ***Olly*** y el programa arranca, un fallo ¿? No lo sé pero arranca eso seguro, si tenemos ***Regmonitor*** o ***Filemonitor***, etoc., no carga, jeje.. detecta su controlador!!

Antes de continuar, este truquito del código encriptado lo podemos abordar desde distintos frentes, a "bote pronto" se me ocurren dos formas de hacerlo: o bien mediante un injerto o bien salvando el archivo con el código ya desencriptado puesto que lo he comprobado con el programa en ejecución, o sea abierto y el código está ahí visible por lo que salvando las modificaciones y nopeando los lugares que intentarán desencriptar puesto que no haría falta al estar ya desencriptado funkaría también. Por ganas de aprender que tenemos lo vamos a hacer de las dos maneras para que sepamos como hacerlo en otros casos si hiciera falta ;-)

Método 1: Inyectando código…

Pues eso mismo imaginaos que tenemos una ***jeringuilla*** con los ***bytes*** preparados, jejeje y tenemos claro un paciente llamado ***CoolPicker*** que necesita esos bytes para funcionar, le haremos un favor y le concederemos el deseo, aquí hay dos cosas a tener en cuenta (y supongo que más también):

1.- Qué código inyectar
2.- Donde hacerlo

No se puede ir a tontas y a locas por ahí porque no coneguiríamos nada, haremos de cirujanos pues, primeramente estudiaremos el código para ver donde deberemos parchear, qué bytes se necesitan cambiar y conseguido el parche digamos al vuelo entonces sí buscar un lugar seguro para inyectar nuestra rutinita que nos haga el tabajito. Sabemos porque antes lo vimos que la nag se evita desde esta parte:

```
:0040BF0A    FFD6            CALL ESI
:0040BF0C    8A85 E8FEFFFF   MOV AL,BYTE PTR SS:[EBP-118]
:0040BF12    6A 00           PUSH 0
:0040BF14    84C0            TEST AL,AL
:0040BF16    0F85 D0000000   JNZ DUMP_.0040BFEC          ; Si salta no vemos la nagscreen..!!
```

Ese ***CALL ESI*** va a la rutina que quedaba dentro de la ***IAT*** y que no reparamos ya que no va a ninguna ***API***, parece una rutina anti-dump pero es una chorrada porque cae dentro del .exe y está en el dumpeado así que no hay problema, tenemos que a la salida del call ***AL*** toma el valor de ***[EBP-118]*** que en ese momento representa ***[12FDF8]*** y cuyo byte bajo es lógicamente un ***0***. Podría pues modificar la rutina y adaptarla a nuestras necesidades, veamos que necesitamos entonces, primero veamos desde donde se llama a la misma:

```
:0040C9E5   FF15 D0B44300   CALL DWORD PTR DS:[43B4D0]   ; ColorPic.005124DB
:0040EA6E   FF15 D0B44300   CALL DWORD PTR DS:[43B4D0]   ; ColorPic.005124DB
:0041112D   FF15 D0B44300   CALL DWORD PTR DS:[43B4D0]   ; ColorPic.005124DB
:0042AB76   FF15 D0B44300   CALL DWORD PTR DS:[43B4D0]   ; ColorPic.005124DB
:00431FE2   FF15 D0B44300   CALL DWORD PTR DS:[43B4D0]   ; ColorPic.005124DB
:0040BF0A   FFD6            CALL ESI
```

Desde estos cinco lugares se llama a esta rutina, pero el injerto no debe modificar todos los retornos por igual porque lo que necesitamos por ejemplo en el ***CALL ESI*** no lo necesitamos en la llamada en ***:40C95E***, esto es un inconveniente a tener muy en cuenta, deberemos pues colocar un chequeo que verifique de donde venimos, pero cómo se hace eso? La respuesta está en la ***PILA*** justo en el ***RET*** final tenemos en ***[ESP]*** la dirección de retorno, que podríamos recuperar y comparar con la dirección que sea y así ser selectivo, en función de que vengamos de un lugar o de otro actuaremos de diferente forma. Probemos va…

- cuando vengamos desde ***:40BF0A*** nos interesaría hacer:

```
:005124DB    C685 E8FEFFFF 01        MOV BYTE PTR SS:[EBP-118],1
```

y así pues que la rutina cambie el valor de ***[EBP-118]*** como necesitamos, además también se debe ejecutar bien para no descuadrar el programa, podria modificar el ***RET*** final y la instrucción siguiente luego la restauraría antes de retornar claro, tengo ***5 bytes*** aquí:

```
:005124EB    C3              RETN
:005124EC    66:BA 7B5C      MOV DX,5C7B
```

justo para ensamblar un salto a una zona libre del exe y que realice el parche en valor de retorno, jeje.. pero no se porque este ***RET*** no me lleva al sitio adecuado sinó a otro, hago un ***tracing*** y veo que el verdadero ***RET*** está aquí:

```
:00512758    FFE0            JMP EAX
:0051275A    C2 1000         RETN 10        ; Verdadero retorno de la función..!!
```

Para injertar me podría beneficiar de estas sentencias y luego restaurarlas:

```
:0051275A    C2 1000         RETN 10        ; Deberemos reataurar este RETN 10 después
:0051275D    52              PUSH EDX       ; Ojo que no nos pertenece
:0051275E    51              PUSH ECX       ; Ojo que no nos pertenece
```

Aquí podemos ensamblar el salto a una zona libre del exe pero con cuidado porque como dicen los comentarios las dos instrucciones finales no nos pertenecen ya que forman parte de otra rutina, si las usamos para el salto las deberemos restaurar antes de retornar para que cuando la rutina esta sea llamada se encuentre los bytes correctos y no falle, el siguiente paso es buscar un lugar libre para nuestro injerto, tenemos de sobra a partir de ***:530300*** entonces sustituimos estas 3 instrucciones por un salto hacia allí:

```
:0051275A   E9 A1DB0100     JMP DUMP_.00530300
```

a partir de ***:530300*** pondremos esto:

```
:00530300    60                          PUSHAD
:00530301    8B4424 20                   MOV EAX,DWORD PTR SS:[ESP+20]
:00530305    3D 0CBF4000                 CMP EAX,DUMP_inj.0040BF0C
:0053030A    74 02                       JE SHORT DUMP_inj.0053030E
:0053030C    61                          POPAD
:0053030D    C3                          RETN
:0053030E    C685 E8FEFFFF 01            MOV BYTE PTR SS:[EBP-118],1
:00530315    C705 5A275100 C2100052      MOV DWORD PTR DS:[51275A],520010C2
:0053031F    C605 5E275100 51            MOV BYTE PTR DS:[51275E],51
:00530326    61                          POPAD
:00530327  ^ E9 2E24FEFF                 JMP DUMP_inj.0051275A
```

Explico un poco lo que hace que es realmente simple, en el ***PUSHAD*** salvo los registros a la ***PILA*** que luego restauraremos al final con el ***POPAD***, obtenemos la dirección de retorno en al ***MOV EAX, [ESP+20]***, esto se sabe mirando al stack estando después del ***PUSHAD*** inicial, veamoslo:

```
ESP ==>   0012FDF9
ESP+4     005124DB  DUMP_inj.005124DB
ESP+8     0012FF10
ESP+C     0012A61C
ESP+10    00000060
ESP+14    0012FDF8
ESP+18    0012FDF8
ESP+1C    0010E000
ESP+20    0040BF0C  DUMP_inj.0040BF0C
ESP+24    FFFFFFFF
```

La zona sombreada marca el valor de retorno ***:40BF0C***, si vamos allí vemos:

```
Address   Hex dump    Disassembly                          Comment
0040BF0A    FFD6       CALL ESI                             Apunta a :5124DB
0040BF0C    8A85 E8FEFFFF  MOV AL,BYTE PTR SS:[EBP-118]
0040BF12    6A 00      PUSH 0
```

Pues la dirección siguiente al ***CALL ESI*** desde onde se hizo la llamada, podemos pues decir que estamos en lo cierto, tras el ***MOV EAX, [ESP+20]*** quedará ***EAX==40BF0C*** por esto se compara con este valor después, si coincide sabemos desde donde se hizo la llamada y podemos intervenir, saltará a ***:53030E*** donde modificará el valor de retorno que va en ***[EBP-118]*** como sabemos e importante restaura las instrucciones que rescribimos con el ***JMP*** en ***:51275A***, o sea que dejaremos todo como estaba, injertemos va:

```
Address   Hex dump    Disassembly                          Comment
00530300    60         PUSHAD
00530301    8B4424 20  MOV EAX,DWORD PTR SS:[ESP+20]
00530305    3D 0CBF4000  CMP EAX,DUMP_.0040BF0C
0053030A  v 74 02      JE SHORT DUMP_.0053030E
0053030C    61         POPAD
0053030D    C3         RETN
0053030E    C685 E8FEFFFF  MOV BYTE PTR SS:[EBP-118],1
00530315    C705 5A275100  MOV DWORD PTR DS:[51275A],520010C2
0053031F    C605 5E275100  MOV BYTE PTR DS:[51275E],51
00530326    61         POPAD
00530327  v E9 00000000  JMP DUMP_.0053032C
0053032C    0000       ADD BYTE PTR DS:[EAX],AL
```

Y salvamos los cambios con ***Copy to Executable->Selection***, podríamos aprovechar para salvar el cambio del salto pero como la opción ***All Modifications*** no la da no sé por qué luego lo haremos, guardamos los cambios como ***DUMP***_injerto.exe así nos referiremos todos al mismo archivo, lo abrimos en otro ***Olly*** y realizamos la misma operación pero para el salto:

```
Address   Hex dump    Disassembly                          Comment
0051275A  v E9 A1DB0100  JMP DUMP_.00530300
0051275F    BA 00000000  MOV EDX,0
00512764    50         PUSH EAX
```

lo salvamos como ***DUMP_injerto_1.exe***, y lo abrimos en este mismo ***Olly***, ya está todo preparado para ver si funciona pongamos un ***BP*** en ***:51275A*** donde está el salto, damos ***RUN*** y allí para traceamos un poco y nos damos cuenta de que van bien las cosas, se restauran bien las insrtucciones reemplazadas por el salto y se modifica el flag también, retornamos, salta en ***:40BF16*** y entramos sin ver la nag inicial, jejeje.. ***FUNKA***!!

Esto funciona chavales, lo mejor de todo es que el ***TimeTrial*** cae directamente ya que por lo visto la rutina que lo mira estará cerca del call que saca la ***nag*** y como el salto evita la zona evita también la limitación, dos pájaros de un disparo, ahora quedan algunos detalles que pulir, si vamos al ***About*** queda un ***UNREGISTERED*** allí, mirad:

```
:0040EA6E  .  FF15 D0B44300    CALL DWORD PTR DS:[43B4D0]
:0040EA74  .  8A4424 18        MOV AL,BYTE PTR SS:[ESP+18]
:0040EA78  .  84C0             TEST AL,AL
:0040EA7A  .  75 12            JNZ SHORT DUMP_inj.0040EA8E
:0040EA7C  .  68 0C824400      PUSH DUMP_inj.0044820C        ;  ASCII "UNREGISTERED"
:0040EA81  .  8D8D DC0C0000    LEA ECX,DWORD PTR SS:[EBP+CDC]
:0040EA87  .  E8 D3D90100      CALL DUMP_inj.0042C45F
:0040EA8C  .  EB 74            JMP SHORT DUMP_inj.0040EB02
:0040EA8E  >  68 FC814400      PUSH DUMP_inj.004481FC        ;  ASCII "Licensed to "
```

Para que muestre nuestro nombre o bien adaptamos el injerto para la ocasión o bien jumpeamos el salto que es lo que yo he hecho, además cambién el ***Licensed to*** por mi nombre en ***[44820C]***, sólo quedaría quitar los botones ***BUY NOW!*** y ***REGISTER!*** Del cuadro principal y no hay que buscar mucho pues se hace aquí:

```
:0040CA01  .  E8 3D280200      CALL DUMP_inj.0042F243
:0040CA06  .  8A4424 44        MOV AL,BYTE PTR SS:[ESP+44]
:0040CA0A  .  84C0             TEST AL,AL
:0040CA0C  .  8B4424 20        MOV EAX,DWORD PTR SS:[ESP+20]
:0040CA10  .  75 09            JNZ SHORT DUMP_inj.0040CA1B        ; Debe saltar..!!
:0040CA12  .  8B5424 34        MOV EDX,DWORD PTR SS:[ESP+34]
:0040CA16  .  8D0C02           LEA ECX,DWORD PTR DS:[EDX+EAX]
:0040CA19  .  EB 08            JMP SHORT DUMP_inj.0040CA23
```

concretamente debemos jumpear elsalto de ***:40CA10***, hecho esto ya tenemos el programa totalmente desprotegido del ***ACProtect*** y las limitaciones, estos últimos cambios se pueden realizar sin modificar nuestro injerto ya que son zonas que siempre están desencriptadas, por lo visto sólo se han molestado en encriptar la zona de creación de la nag y verificación del ***Trial*** ;DDD

Llegados a este punto tenemos el programa totalmente crackeado no hay referencias a Textos molestos y nos reconoce en el About debido al cambiazo de la cadena que hicimos antes, además entramos sin la nag y sin limitación de días, diría yo que el objetivo está cubierto.

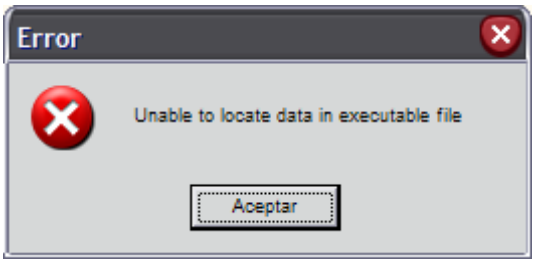
El tuto podría acabar aquí pero os quiero mostrar una manera muy muy sencilla de resolver esto sin necesidad de injertar que pasaré a describir a continuación:

Método 2: Venciendo rutinas de desencriptación..

Mirad que fácil, coger el exe dumpeado que no tiene injerto y por tanto sale la ***nag*** y todo eso, lo cargamos en ***Olly*** y damos ***RUN*** dejamos que cargue totalmente el programa, seleccionamos todos los bytes de la sección .text que es donde estaba el código encriptado:

```
Address   Hex dump                                          ASCII
0043AE90  FF FF E9 90 C4 FF FF B8 B8 57 44 00 E9 CF EB FD
0043AEA0  FF CC CC CC FF 75 F0 E8 5A B3 FF FF C3 B8 18 58
0043AEB0  44 00 E9 B9 EB FD FF 00 00 00 00 00 00 00 00 00  D.
0043AEC0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ....
0043AED0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ....
```

debemos dejar los ceros sin seleccionar sino aparecerá este ***cartel***:



Dejando seleccionada la zona como en la imagen anterior a esta no hay problema y podemos salvar, lo llamamos por ejemplo ***7rutinas.exe***, si lo ejecutamos saltará un error o sea que no funciona, lógico pues tiene ***7 rutinas desencriptadoras*** que tratarán de desencriptar unos bytes que ya están desencriptados, lo que debemos hacer es nopear estos ***7 lugares*** donde quiere escribir y que son:

```
:00408DA6   890B       MOV DWORD PTR DS:[EBX],ECX
:00408EF1   8937       MOV DWORD PTR DS:[EDI],ESI
:0040904A   8928       MOV DWORD PTR DS:[EAX],EBP
:0040B97C   8930       MOV DWORD PTR DS:[EAX],ESI
:0040BAD2   8932       MOV DWORD PTR DS:[EDX],ESI
:0040BC51   8955 00    MOV DWORD PTR SS:[EBP],EDX
:0040BDB8   8928       MOV DWORD PTR DS:[EAX],EBP
```

Iremos nopeando uno por uno:

```
Address   Hex dump   Disassembly                      Comment
00408DA6  890B       MOV DWORD PTR DS:[EBX],ECX
00408DA8  EB 01      JMP SHORT DUMP_inj.00408DAB
00408DAA  E8         DB E8
```

lo dejaremos de este modo:

```
Address   Hex dump   Disassembly                      Comment
00408DA6  90         NOP
00408DA7  90         NOP
00408DA8  EB 01      JMP SHORT DUMP_inj.00408DAB
```

Hacemos esto mismo con los otros ***6*** y salvamos los cambios, lo ejecutamos y funciona también con la diferencia de que ahora el ***CALL ESI*** está accesible ya desde el ***OEP***, jejeje… podemos pues superar el ***Trial*** y la ***nag*** cambiando el siguiente salto en lugar de toquetear el valor de ***[EBP-118]*** como antes hacíamos:

```
Address   Hex dump     Disassembly                    Comment
0040BF14  .  84C0      TEST AL,AL
0040BF16  v  E9 D1000000  JMP DUMP_pr_.0040BFEC
0040BF1B     90        NOP
0040BF1C  .  8D8D 80EAFFFF LEA ECX,DWORD PTR SS:[EBP-1580]
0040BF22  .  E8 894F0000  CALL DUMP_pr_.00410EB0
0040BF27  .  BB 01000000  MOV EBX,1
```

Lógicamente no necesitamos injerto alguno porque ahorá si esta zona está accesible siempre por lo que el ***Método 2*** es también válido, probadlo veréis como funciona también.

Y hasta aquí hemos llegado, mi idea era mostrar o más bien explicar la manera que utilicé para desempaquetar y luego superar el escollo de los calls desencriptadores, ciertamente he pasado un buen rato, espero que vosotros también lo hayáis disfrutado para eso lo hago además de para ayudar a todo aquél que empieza en este apasionante mundo del ***cracking***, mis fuerzas para ellos y estaré en ayudarles ***SIEMPRE***..!!

Por fin sólo saludar a toda la peña de ***[CracksLatinos]*** que la verdad es estupenda ***[ZettK]*** nos veremos las caras con el ***ASProtect 2.0*** socio, lo mismo te digo ***+NCR***, un especial saludo para ellos y para ***Ricardo*** que siempre me está echando cables cuando lo necesito, gracias amigo, este tuto está dedicado para ellos en especial porque no merecen menos, no sin olvidarme del resto que no los conozco aún pero sé de sobras que son igual así que para ellos también va..

Aupa ***CracksLatinos***!!

Si alguien se quiere poner en contacto conmigo por alguna duda de este tuto o por otra que no sea de este puede hacerlo enviandome un mensaje a mi correo electrónico:

syxe00@yahoo.es
syxe00@hotmail.com

Os pongo los dos pero preferiría recibrlos por yahoo que me es más cómodo. También espero ayuda para el tema de ***SafeDisc***, es la versión "***2.67.010***"según ***PEiD***, hay otros con la "***2.65.010***" por si alguien quiere ir mirando el tema les paso el nombre de las aplicaciones protegidas con este ***packer***:

Adobe Acrobat Reader 7.0 => versión 2.65.010
Adobe Illustrator CS2 Tryout => con 2.65.010

Puede bajarse el programa de la propia página de Adobe en:

http://www.adobe.com/products

allí los encontraréis sin mayor complicación, por cualquier cosa me lo pedís a mi y ya está yo os lo pasaré. Espero vuestra ayuda y también ofrezco la mía en lo que necesitéis ¿ok? Un saludo a todos, hasta la próxima..!!